MAKALAH

SMART-RETAIL

Disusun Untuk Memenuhi Tugas Mata Kuliah Logika Pemrograman

Dosen Pengampu:

Dr. Ir. Chaerur Rozikin, S.Kom., M.Kom.



Oleh Kelompok :

Dion Immanuel Putra	2610631140159
Luna Maulidina	2610631140157
Reza Ahmad Sodikin	2610631140160

PROGRAM STUDI INDUSTRI

FAKULTAS TEKNIK

UNIVERSITAS SINGAPERBANGSA KARAWANG

TAHUN AJARAN 2026/2027

Daftar Isi

Daftar Isi ii
Daftar Gambar iii
1. Latar Belakang 1
Tujuan 1
Fitur Utama ..... **Error! Bookmark not defined.**
2. Flowchart Sistem 2
2.1 Flowchart Alur Utama Sistem 2
2.2 Flowchart Proses Transaksi Penjualan 3
2.3 Flowchart Kelola Produk 4
3. ERD Database 5
3.1 Kamus Data 6
4. Penjelasan Program 7
4.1 Struktur dan Teknologi 7
4.2 Fungsi di Luar Kelas 7
4.3 Method pada Kelas SmartRetail 7
4.4 Penjelasan Logika Penting 8
4.5 Akun Awal dan Cara Menjalankan 9
5. Screenshot Program 10
6. Pengujian Sistem 15
7. Kesimpulan 18
Saran Pengembangan 18

Daftar Gambar

Gambar 2.1 Flowchart alur utama (login, hak akses, dan logout) 2
Gambar 2.2 Flowchart transaksi penjualan oleh kasir 3
Gambar 2.3 Flowchart kelola produk oleh admin 4
Gambar 3.1 Entity Relationship Diagram (notasi crow's foot; PK = primary key, FK = foreign key) 5
Gambar 5.1 Halaman login 10
Gambar 5.2 Dashboard (Admin) 10
Gambar 5.3 Kelola Produk (Admin) 11
Gambar 5.4 Dialog Tambah Produk 11
Gambar 5.5 Riwayat Transaksi (Admin) 12
Gambar 5.6 Laporan Penjualan (Admin) 13
Gambar 5.7 Dashboard (Kasir) 13
Gambar 5.8 Transaksi Penjualan dengan keranjang (Kasir) 14

1. Latar Belakang

Masalah yang sering dialami toko kelontong itu rata-rata ada di pencatatan barang dan uang yang masih manual. Kalau cuma ngandelin buku catatan atau ingatan, rawan banget salah hitung. Dampaknya kelihatan pas ada barang di rak yang hilang atau selisih, tapi pemilik toko bingung ngetrace-nya dari mana. Selain itu, pemilik juga susah tau pasti barang apa sih yang sebenarnya paling laku, atau berapa keuntungan bersih yang mereka dapat tiap bulannya. Pas lagi ramai pembeli, kasir juga butuh waktu lebih lama buat ngitung belanjaan dan kembalian secara manual.

Melihat kondisi itu, dibangunlah aplikasi Smart Retail. Aplikasi kasir berbasis desktop ini dibuat pake bahasa Python. Tampilannya memanfaatkan pustaka Tkinter, sementara untuk databasenya pakai SQLite (smart_retail.db) supaya bisa berjalan lokal tanpa butuh koneksi internet atau server tambahan. Di aplikasi ini ada dua hak akses pengguna: Admin yang bertugas ngelola data barang dan ngeliat laporan penjualan, sama Kasir yang fokus ngelayanin transaksi pembayaran.

Tujuan

1. Menerapkan konsep logika pemrograman (kayak percabangan, perulangan, fungsi, validasi input, dan struktur data) ke bentuk aplikasi nyata.
2. Membantu otomatisasi perhitungan belanjaan, kembalian, sekaligus pemotongan stok barang.
3. Menyimpan data barang dan transaksi secara rapi di database relasional.
4. Membatasi hak akses fitur sesuai peran pengguna (admin dan kasir).

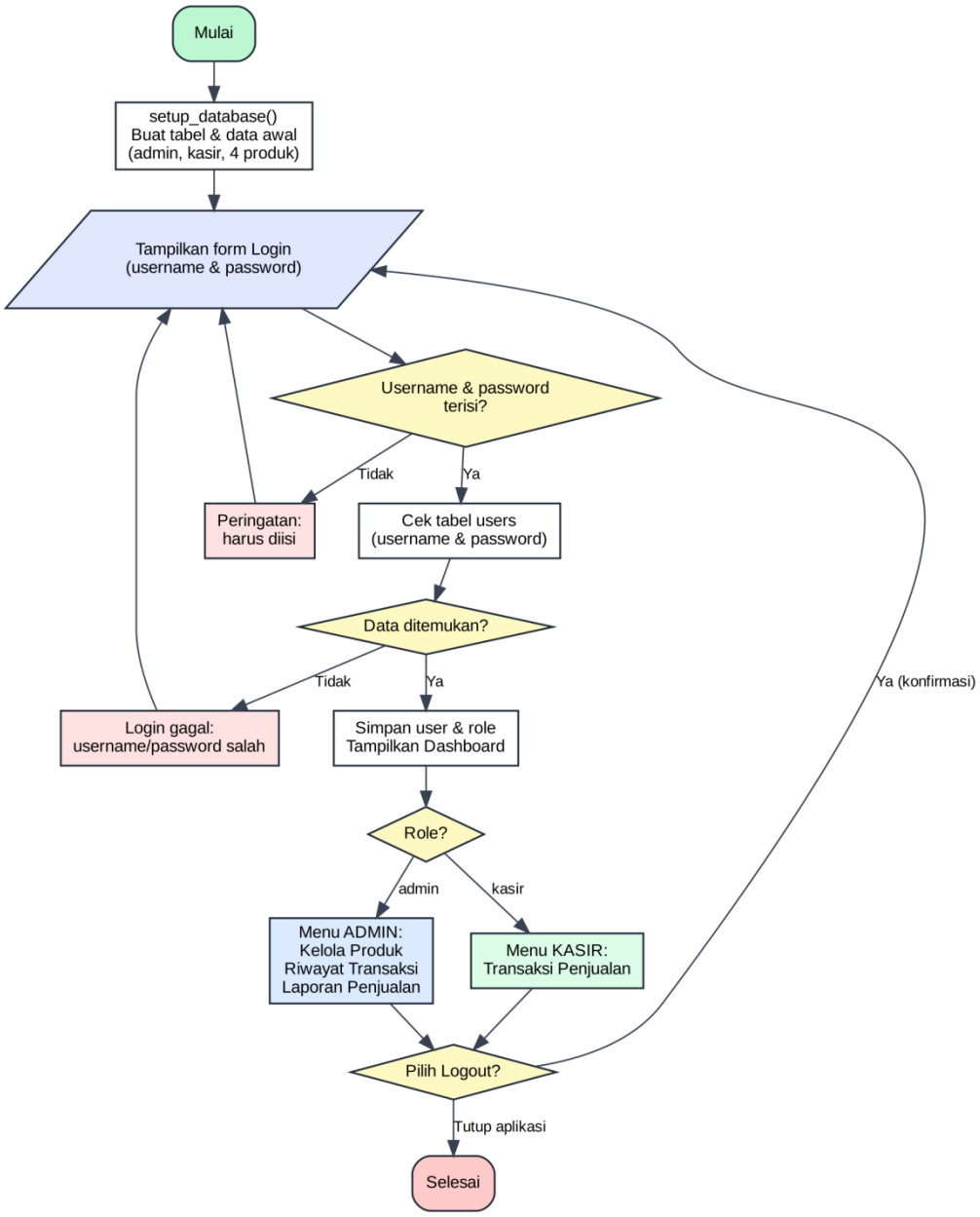
Fitur Utama

1. Login terpisah sesuai peran (Admin & Kasir).
2. Dashboard ringkasan (total produk, stok, jumlah transaksi, dan pendapatan).
3. Kelola Produk khusus Admin (tambah, edit, hapus, dan cari barang).
4. Transaksi Penjualan untuk Kasir (keranjang, pembayaran, kembalian, dan cetak struk).
5. Riwayat Transaksi & Laporan Penjualan buat ngecek omzet dan barang terlaris.

2. Flowchart Sistem

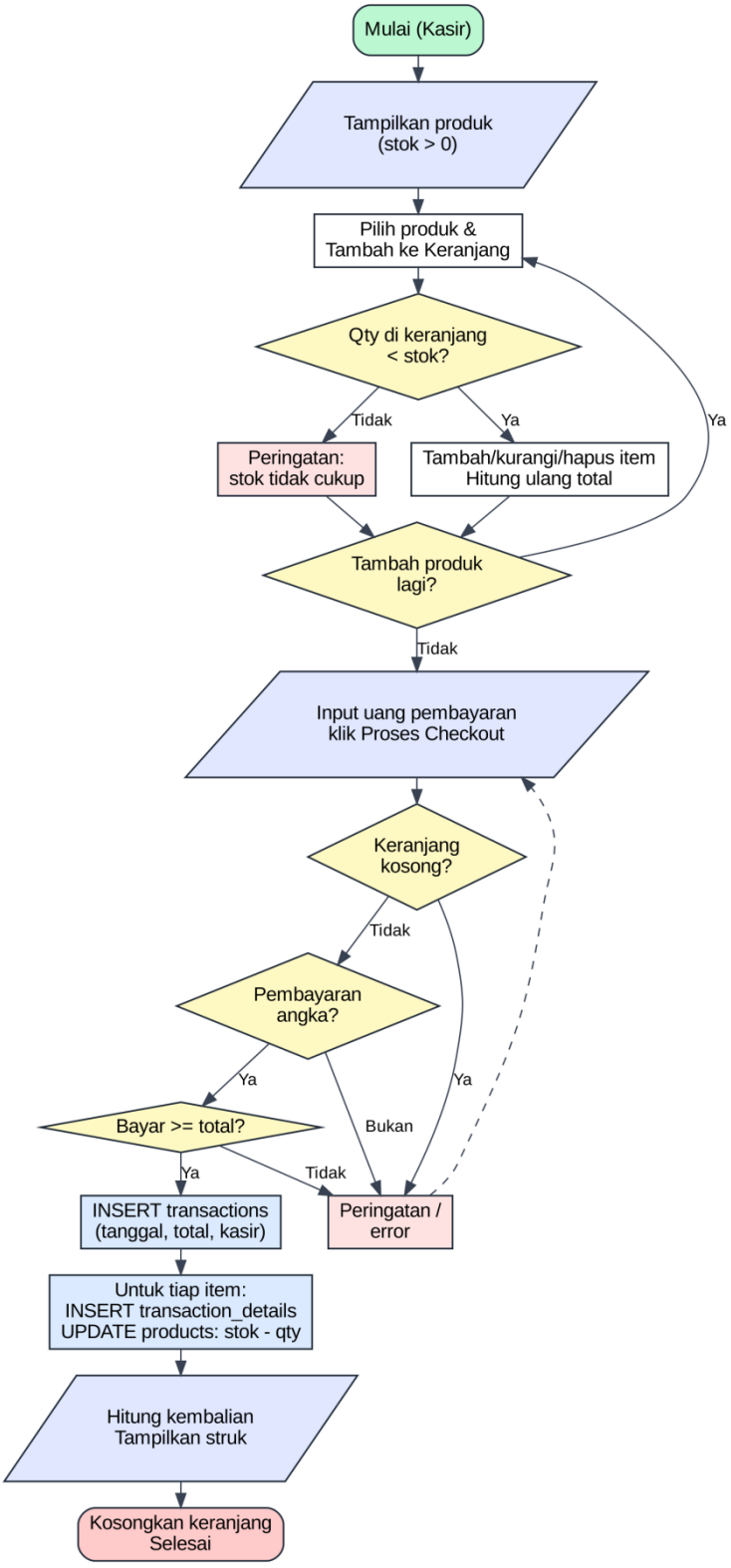
Flowchart berikut menggambarkan alur kerja program. Bagian 2.1 menunjukkan alur utama dari menjalankan aplikasi, login, pembagian peran, hingga logout. Bagian 2.2 dan 2.3 merinci dua proses penting: transaksi penjualan oleh kasir dan pengelolaan produk oleh admin.

2.1 Flowchart Alur Utama Sistem



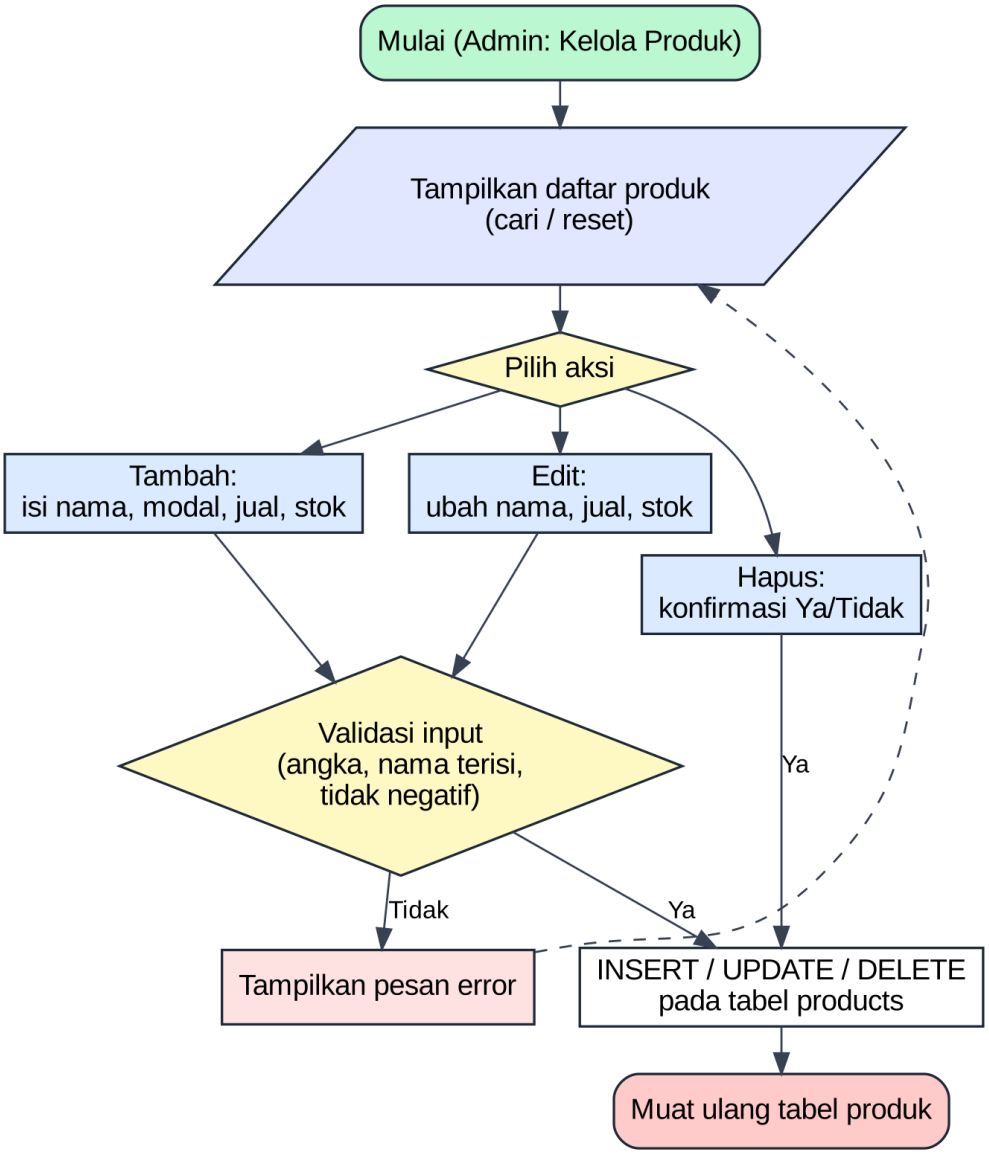
Gambar 2.1 Flowchart alur utama (login, hak akses, dan logout)

2.2 Flowchart Proses Transaksi Penjualan



Gambar 2.2 Flowchart transaksi penjualan oleh kasir

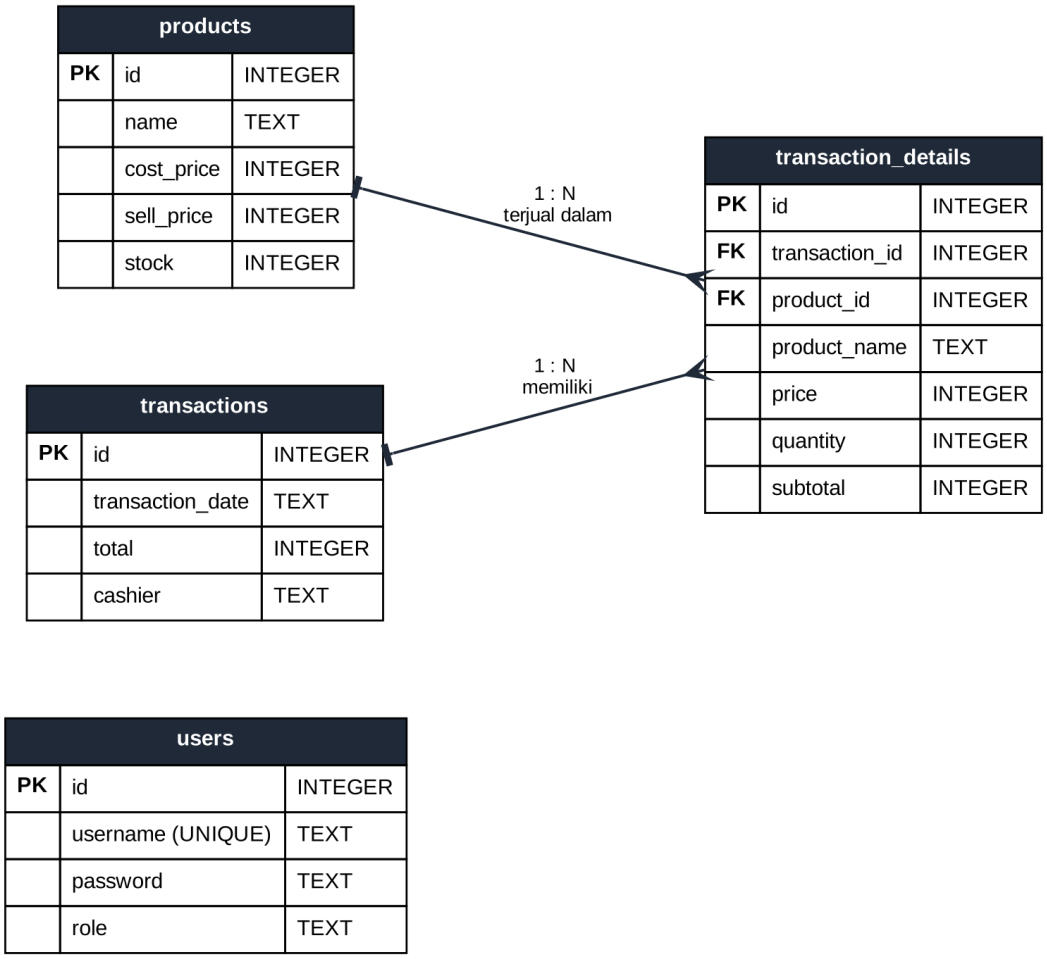
2.3 Flowchart Kelola Produk



Gambar 2.3 Flowchart kelola produk oleh admin

3. ERD Database

Database **smart_retail.db** terdiri dari empat tabel. Tabel *transaction_details* menghubungkan tabel *transactions* dan *products*: satu transaksi dapat memiliki banyak detail barang, dan satu produk dapat muncul pada banyak detail transaksi (relasi satu-ke-banyak). Tabel *users* berdiri sendiri; nama kasir pada transaksi disimpan sebagai teks di kolom *cashier*.



Gambar 3.1 Entity Relationship Diagram (notasi crow's foot; PK = primary key, FK = foreign key)

3.1 Kamus Data

Tabel	Kolom	Tipe	Keterangan
users	id	INTEGER PK	ID pengguna, otomatis bertambah
	username	TEXT UNIQUE	Nama login, tidak boleh sama
	password	TEXT	Kata sandi
	role	TEXT	Peran: admin / kasir
products	id	INTEGER PK	ID produk
	name	TEXT	Nama barang
	cost_price	INTEGER	Harga modal (Rp)
	sell_price	INTEGER	Harga jual (Rp)
	stock	INTEGER	Jumlah stok
transactions	id	INTEGER PK	ID transaksi
	transaction_date	TEXT	Tanggal & waktu (YYYY-MM-DD HH:MM:SS)
	total	INTEGER	Total belanja (Rp)
	cashier	TEXT	Username kasir yang melayani
transaction_details	id	INTEGER PK	ID detail
	transaction_id	INTEGER FK	Mengacu ke transactions(id)
	product_id	INTEGER FK	Mengacu ke products(id)
	product_name	TEXT	Nama produk saat transaksi
	price	INTEGER	Harga satuan saat transaksi
	quantity	INTEGER	Jumlah dibeli
	subtotal	INTEGER	price × quantity

Kolom *product_name* dan *price* pada detail transaksi sengaja menyimpan salinan nama dan harga saat transaksi terjadi, sehingga riwayat tidak berubah walaupun produk kemudian diedit.

4. Penjelasan Program

4.1 Struktur dan Teknologi

Aplikasi ini dibuat dalam satu file kode Python yaitu Smart_Retail_full_code.py dengan panjang sekitar 2.600 baris, ditambah file database smart_retail.db. Beberapa pustaka yang dipakai di antaranya:

- tkinter & ttk: Buat ngebangun tampilan visual kayak window, button, tabel daftar barang (Treeview), dan kotak pesan.
- sqlite3: Buat ngoperasikkan query database SQLite.
- datetime: Buat nyatet tanggal sama jam transaksi secara otomatis.
- os: Buat nempatin lokasi penyimpanan file database supaya tetep satu folder sama file kodenya.

Secara struktur, kodenya menerapkan konsep Pemrograman Berorientasi Objek (OOP). Kelas SmartRetail megang kendali untuk logika utama dan tampilan antarmuka, sedangkan fungsi-fungsi di luar kelas dipakai buat urusan database dan format angka rupiah.

4.2 Fungsi di Luar Kelas

Fungsi	Penjelasan
get_connection()	Membuka koneksi ke smart_retail.db.
setup_database()	Membuat 4 tabel jika belum ada (CREATE TABLE IF NOT EXISTS); menambah akun awal admin dan kasir dengan INSERT OR IGNORE; mengisi 4 produk awal bila tabel produk masih kosong.
rupiah(angka)	Mengubah angka menjadi format rupiah, misalnya 75000 menjadi Rp75.000.

4.3 Method pada Kelas SmartRetail

Method	Penjelasan
__init__	Membuat jendela utama (1200×700), menyiapkan current_user, current_role, dan list cart (keranjang), lalu menampilkan halaman login.
clear_window	Menghapus semua widget agar halaman dapat berganti.

Method	Penjelasan
show_login, login	Menampilkan form login. login() memvalidasi isian tidak kosong, lalu mencocokkan username dan password pada tabel users (query berparameter); bila cocok menyimpan user dan role lalu membuka dashboard.
show_dashboard, create_sidebar, menu_button	Membuat sidebar. Menu ditentukan oleh role: admin mendapat Kelola Produk, Riwayat Transaksi, Laporan Penjualan; kasir mendapat Transaksi Penjualan.
create_dashboard_content, dashboard_card	Menghitung COUNT/SUM dari database dan menampilkannya dalam 4 kartu ringkasan.
show_products, load_products, search_product	Menampilkan tabel produk; pencarian memakai WHERE name LIKE ?.
add_product, edit_product, delete_product	CRUD produk (khusus admin) dengan jendela dialog dan validasi input: harus angka, nama wajib diisi, nilai tidak boleh negatif (saat tambah); hapus meminta konfirmasi.
show_transaction, load_transaction_products	Halaman kasir: daftar produk yang stoknya > 0 dan panel keranjang.
add_to_cart, increase_cart, decrease_cart, remove_from_cart, refresh_cart	Mengelola keranjang. Jumlah item tidak boleh melebihi stok yang tersedia; total dihitung ulang setiap perubahan.
process_payment	Memvalidasi pembayaran, menyimpan transaksi, menyimpan detail, mengurangi stok, lalu menampilkan struk.
show_history, show_report	Admin melihat daftar transaksi (ORDER BY id DESC) serta laporan: jumlah transaksi, total pendapatan, produk terlaris, dan penjualan per produk (GROUP BY).
logout	Meminta konfirmasi, mengosongkan sesi, kembali ke login.

4.4 Penjelasan Logika Penting

a. Login dan hak akses. Setelah login berhasil, peran disimpan di *current_role*. Selain disembunyikan dari sidebar, halaman admin juga memeriksa peran sebelum dibuka sehingga kasir yang memanggilnya akan ditolak:

```
if self.current_role != "admin":
    messagebox.showerror("Akses Ditolak",
        "Halaman Kelola Produk hanya dapat diakses oleh Admin.")
    return
```

b. Keranjang belanja. Keranjang adalah *list* berisi *dictionary* (id, nama, harga, quantity, stok). Saat produk yang sama ditambahkan lagi, program hanya menaikkan quantity selama masih di bawah stok (percabangan if), dan total dihitung dengan perulangan atas seluruh isi keranjang.

c. Pembayaran dan penyimpanan transaksi. process_payment() melakukan validasi berlapis: keranjang tidak kosong, uang berupa angka (try/except ValueError), dan uang ≥ total. Bila lolos, kembalian = uang − total, lalu data disimpan:

```
INSERT INTO transactions (transaction_date, total, cashier) VALUES (?, ?, ?)
-- untuk setiap item di keranjang (perulangan for):
INSERT INTO transaction_details (...) VALUES (...)
UPDATE products SET stock = stock - ? WHERE id = ?
```

d. Laporan penjualan. Produk terlaris diperoleh dari query yang mengelompokkan detail transaksi per produk dan mengurutkannya dari jumlah terbesar:

```
SELECT product_name, SUM(quantity), SUM(subtotal)
FROM transaction_details
GROUP BY product_id, product_name
ORDER BY SUM(quantity) DESC
```

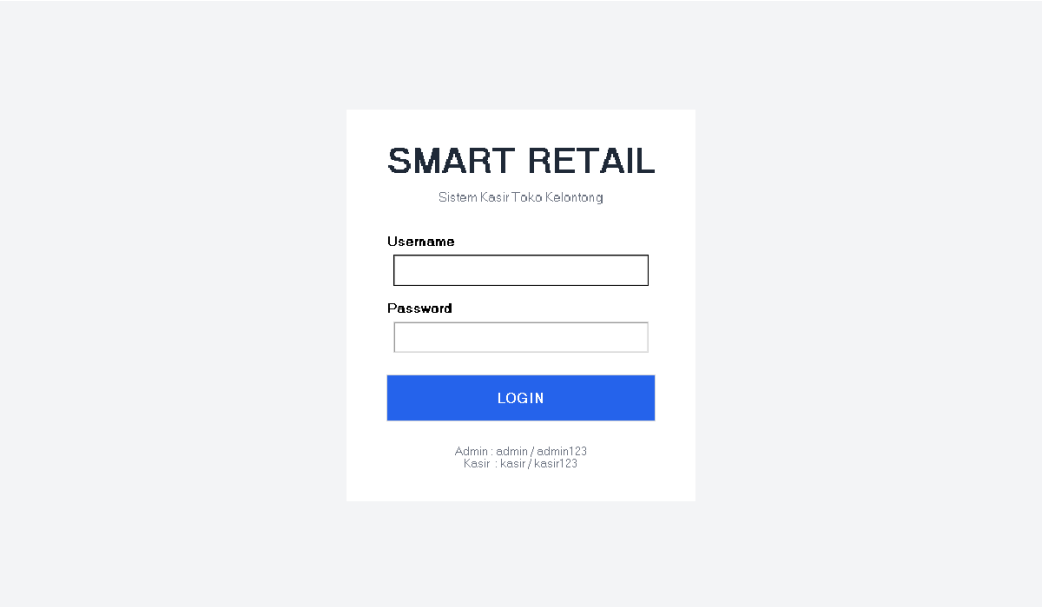
4.5 Akun Awal dan Cara Menjalankan

Peran	Username	Password
Admin	admin	admin123
Kasir	kasir	kasir123

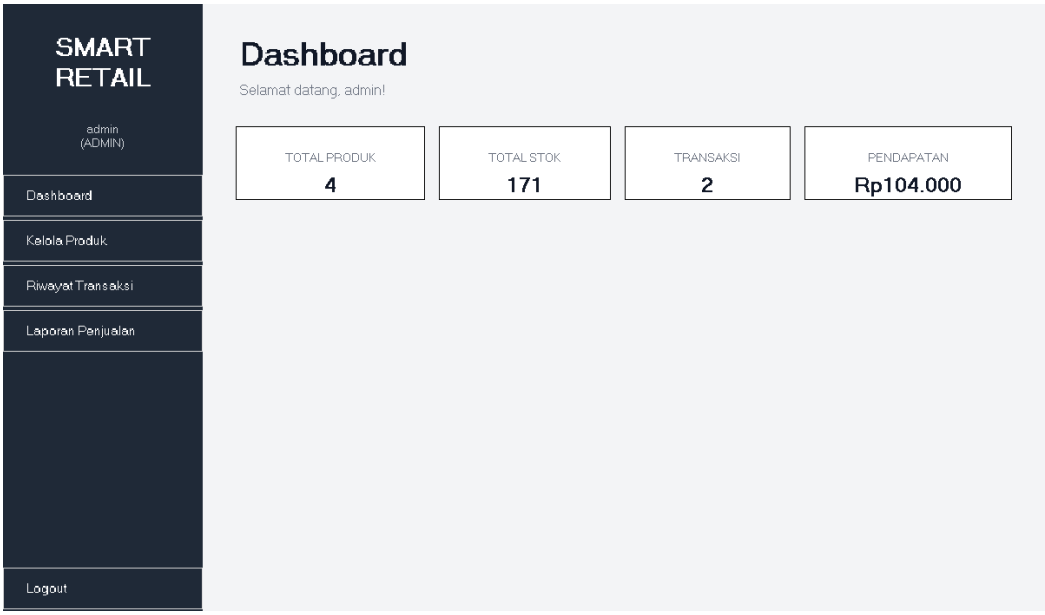
Jalankan dengan perintah python Smart_Retail_full_code.py. Pastikan file smart_retail.db berada di folder yang sama; bila belum ada, database akan dibuat otomatis.

5. Screenshot Program

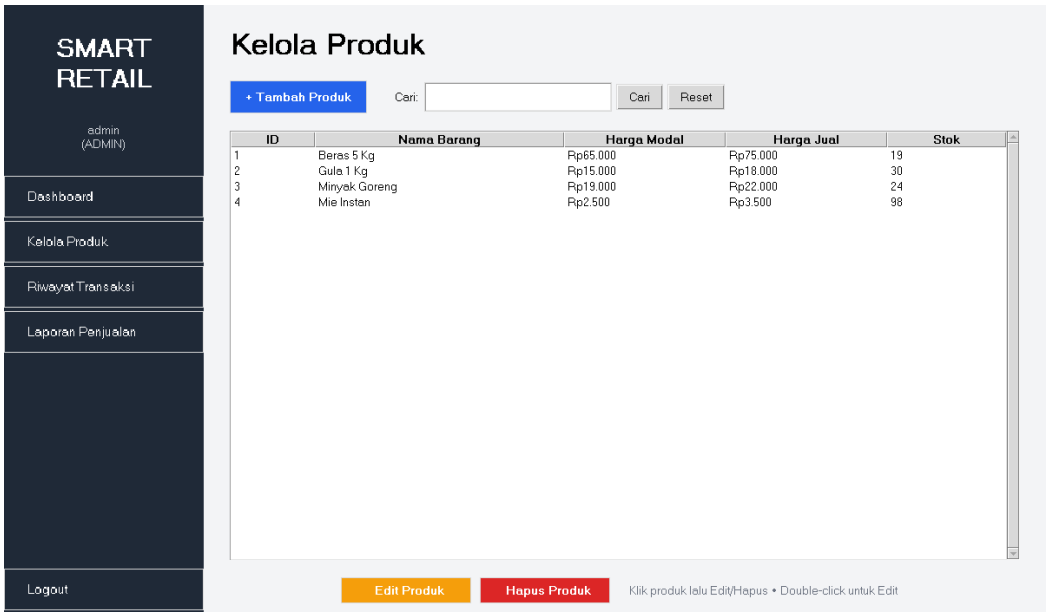
Berikut tampilan setiap halaman aplikasi.



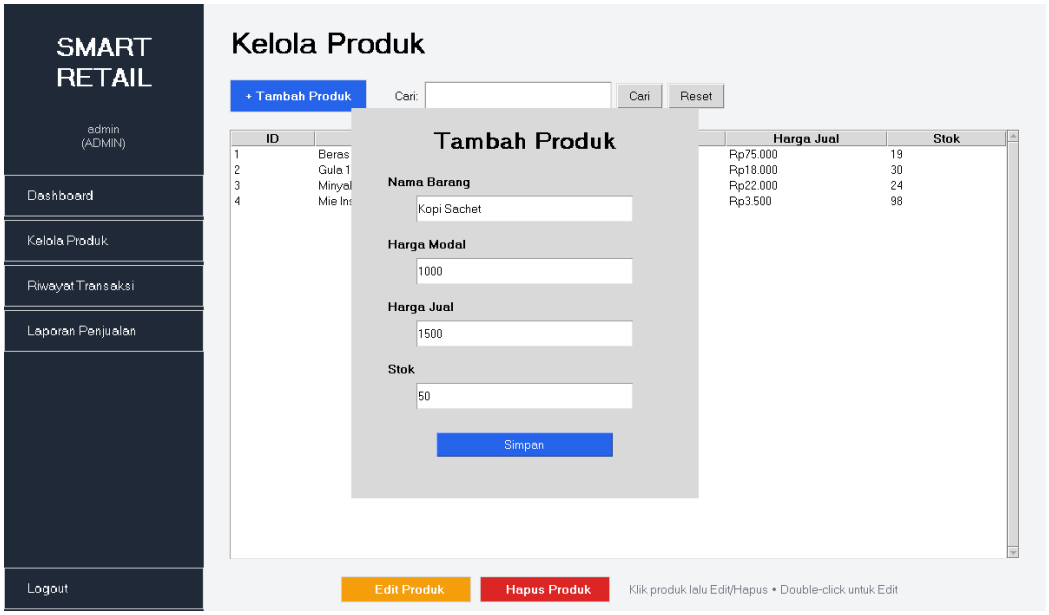
Gambar 5.1 Halaman login



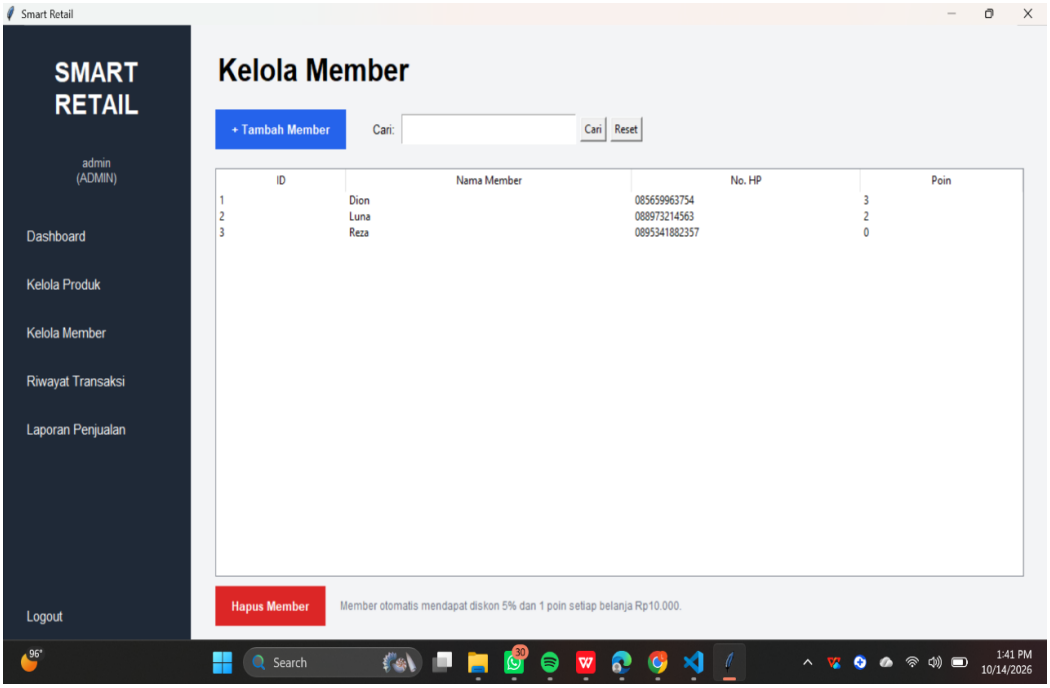
Gambar 5.2 Dashboard (Admin)



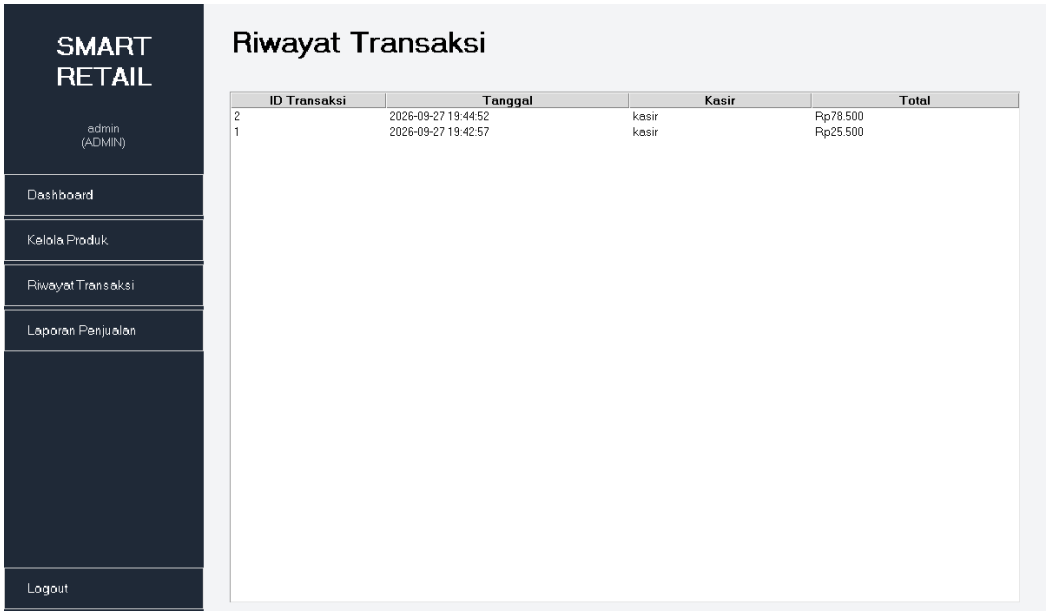
Gambar 5.3 Kelola Produk (Admin)



Gambar 5.4 Dialog Tambah Produk



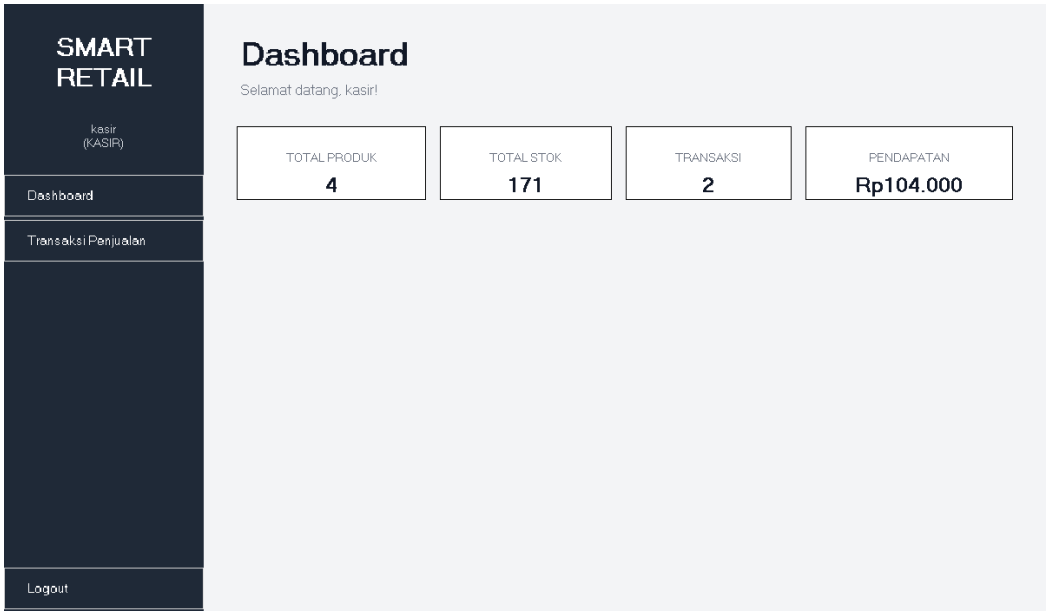
Gambar 5.5 Kelola Member (Admin)



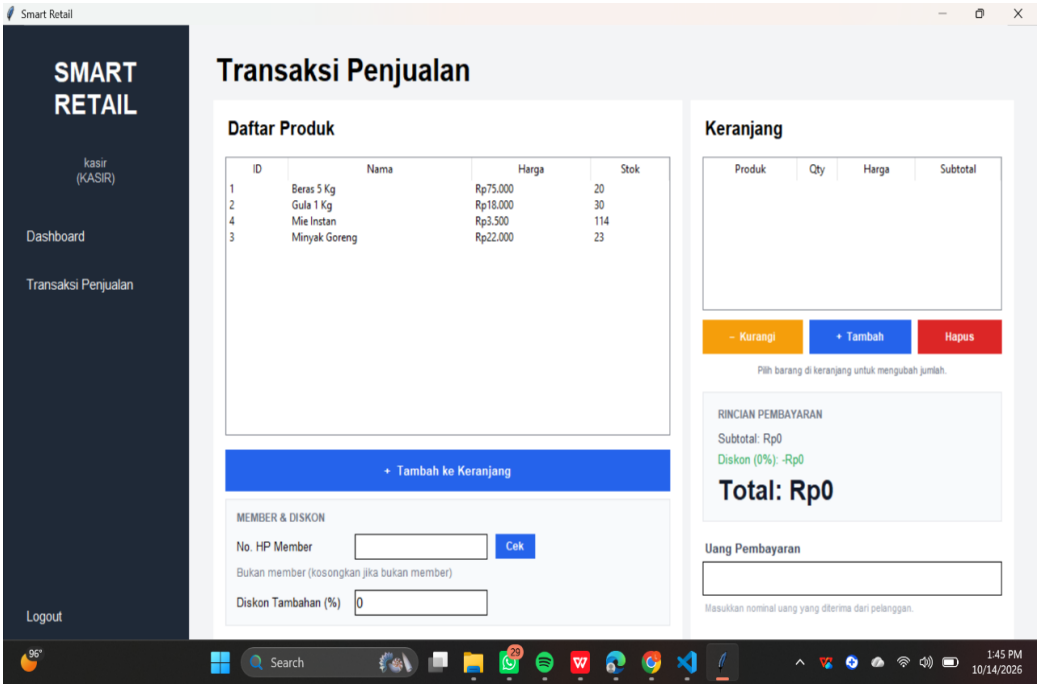
Gambar 5.6 Riwayat Transaksi (Admin)



Gambar 5.7 Laporan Penjualan (Admin)



Gambar 5.8 Dashboard (Kasir)



Gambar 5.9 Transaksi Penjualan

6. Pengujian Sistem

Pengujian dilakukan dengan menjalankan program asli secara otomatis pada **salinan** database baru (bukan database asli), lalu hasil setiap skenario dicatat. Metode yang digunakan adalah *black box testing*: input diberikan, lalu keluaran dibandingkan dengan hasil yang diharapkan.

No	Skenario	Data Uji	Hasil yang Diharapkan	Hasil Pengujian	Status
1	Login admin	admin / admin123	Masuk, menu admin tampil	role=admin, pesan=None	Berhasil
2	Login kasir	kasir / kasir123	Masuk, menu kasir tampil	role=kasir	Berhasil
3	Login password salah	admin / salah	Pesan 'Username atau password salah'	('Login Gagal', 'Username atau password salah.')	Berhasil
4	Login kosong	(kosong)	Peringatan harus diisi	('Peringatan', 'Username dan password harus diisi.')	Berhasil
5	Kasir membuka Kelola Produk	role kasir	Akses ditolak	('Akses Ditolak', 'Halaman Kelola Produk hanya dapat diakses oleh Admin.')	Berhasil
6	Tambah produk valid	Kopi Sachet, 1000, 1500, 50	Produk tersimpan	jumlah produk 4->5; Produk berhasil ditambahkan.	Berhasil
7	Tambah produk stok bukan angka	stok = abc	Pesan error angka	Harga dan stok harus berupa angka.; jumlah tetap 5	Berhasil
8	Tambah produk nama kosong	nama kosong	Peringatan nama wajib	Nama barang harus diisi.	Berhasil
9	Tambah produk harga negatif	modal = -1	Peringatan tidak boleh negatif	Nilai tidak boleh negatif.	Berhasil
10	Cari produk	keyword 'beras'	Hanya Beras 5 Kg tampil	['Beras 5 Kg']	Berhasil
11	Edit produk	Kopi Sachet: harga jual 2000, stok 40	Data berubah di database	sell_price,stock = (2000, 40); Produk berhasil diperbarui.	Berhasil

No	Skenario	Data Uji	Hasil yang Diharapkan	Hasil Pengujian	Status
12	Hapus produk	Kopi Sachet	Produk terhapus	sisa 4 produk; Produk berhasil dihapus.	Berhasil
13	Checkout keranjang kosong	keranjang kosong	Peringatan keranjang kosong	Keranjang masih kosong.	Berhasil
14	Tambah ke keranjang	2x Mie Instan, 1x Gula	Total = 2*3500+18000 = 25000	total keranjang 25000	Berhasil
15	Pembayaran bukan angka	bayar = abc	Pesan error angka	Pembayaran harus berupa angka.	Berhasil
16	Pembayaran kurang	total 25000, bayar 10000	Ditolak, tampil kekurangan	Uang kurang Rp15.000	Berhasil
17	Qty melebihi stok	tambah qty terus-menerus (stok Mie Instan=100)	Ditolak saat qty = stok	qty akhir 100; Stok Mie Instan hanya tersedia 100 pcs.	Berhasil
18	Checkout berhasil	total 25000, bayar 50000	Transaksi+detail tersimpan, stok berkurang, struk tampil	transaksi 0->1, detail 0->2, stok Mie Instan 100->98, total tersimpan 25000	Berhasil
19	Admin melihat riwayat & laporan	setelah transaksi baru	Halaman tampil tanpa error	OK (tidak ada exception)	Berhasil

Hasil: **19 dari 19** skenario berhasil.

Contoh struk yang dihasilkan pada pengujian checkout (No. 18):

```
╔═══════════════════════════════════════╗
║         SMART RETAIL          ║
║        STRUK PEMBELIAN        ║
╠═══════════════════════════════════════╣
ID Transaksi : 1
Tanggal      : 2026-10-13 01:09:38
Kasir        : kasir
╠═══════════════════════════════════════╣
```

Mie Instan

2 x Rp3.500 = Rp7.000

Gula 1 Kg

1 x Rp18.000 = Rp18.000

TOTAL : Rp25.000

BAYAR : Rp50.000

KEMBALIAN : Rp25.000

7. Kesimpulan & saran

Aplikasi Smart Retail berhasil dibuat dan berjalan sesuai rencana sebagai sistem kasir toko kelontong. Fitur-fiturnya mulai dari login multi-user, kelola stok, perhitungan transaksi otomatis, pemotongan stok, sampai laporan penjualan bisa berfungsi dengan baik. Penggunaan SQLite dengan empat tabel terelasi juga terbukti efektif buat menampung data secara konsisten. Dari hasil pengujian black box, aplikasi mampu menangani skenario normal maupun kesalahan input kayak salah password, input bukan angka, uang kurang, atau stok ga cukup. Proyek ini jadi sarana praktik buat nerapin dasar-dasar logika pemrograman dan ilmu komputer, mulai dari variabel, struktur data, pengondisian, perulangan, penanganan error, OOP, sampai pengelolaan basis data.

Saran Pengembangan

1. Password sebaiknya dienkripsi pakai algoritma hashing (seperti bcrypt atau hashlib), bukan disimpan dalam bentuk teks biasa.
2. Mengaktifkan relasi foreign key di SQLite (PRAGMA foreign_keys = ON) biar integritas data antar-tabel lebih terjaga.
3. Nambahin fitur cetak struk ke PDF/printer fisik, filter laporan per rentang tanggal, sama notifikasi kalau stok barang udah mulai tipis.